

Opynio Widget Loader — v6.1.2

Carga perezosa SEO-safe + reservación de espacio + timers visibility-aware (con fixes de revisión independiente)

v6.0 → v6.1.2

Por qué se hizo

El loader v6.0 inicializaba **todos** los widgets en `DOMContentLoaded`, sin importar si estaban dentro o fuera del viewport del host. Cada init disparaba 2 round-trips de red (Supabase + Google Translate) en paralelo al render del cliente. Resultado: degradación del LCP/INP del host y riesgo de penalización por Core Web Vitals.

Cambios aplicados (público: `public/widget.js`)

Componente	v6.0	v6.1.2
Trigger de inicialización	Todos a la vez en <code>DOMContentLoaded</code>	<code>IntersectionObserver</code> con <code>rootMargin: '200px 0px'</code> por widget
Defer post-LCP del host	—	<code>requestIdleCallback(timeout: 1500)</code> antes de cada init
Bypass para crawlers	Sólo render interno	Scheduler respeta <code>IS_BOT</code> → render inmediato sin observer
Reservación de espacio (CLS)	—	<code>min-height</code> por tipo aplicado antes de hidratar
Timers de carruseles	<code>setInterval</code> eternos	<code>visibilityAwareInterval</code> : pausa al salir de viewport y con <code>document.hidden</code> ; teardown completo en <code>stop()</code> (IO + listener + interval)
MutationObserver	<code>document.body subtree</code> , dispara siempre	Filtra <code>.opynio-widget</code> , debounce 50 ms; limpia recursos en <code>removedNodes</code> (SPAs)

Fixes post-revisión (v6.1 → v6.1.2)

v6.1.1 — leaks detectados por revisión independiente

- **Leak de listeners `visibilitychange`**: cada re-arm del ticker dejaba un listener pegado a `document`. `stop()` ahora hace `removeEventListener`.
- **Leak de `IntersectionObserver` internos**: cada re-arm creaba un IO nuevo sin desconectar el anterior. `stop()` ahora hace `io.disconnect()`.
- **Cleanup en SPAs**: el `MutationObserver` ahora itera `removedNodes` y libera `lazyObserver` + ticker asociado.
- **Race resuelto** entre IO y `visibilitychange`: `tick()` filtra, el listener llama `start()` incondicional.
- **Test page**: `document.hidden` restaurado con `delete`; checks rAF-throttled durante scroll.

v6.1.2 — endurecimiento adicional

- **BOT_REGEX ampliado**: añadidos `Mediapartners-Google` (AdSense), `Chrome-Lighthouse` (PageSpeed Insights), `TelegramBot` y `Pinterest`. Lighthouse en cliente ya no califica al host con un estado lazy a medio cargar.
- **reserveSpace respeta CSS externo**: lee `getComputedStyle(el).minHeight` antes de aplicar el default, así un host con `.opynio-widget { min-height: 200px }` en su CSS gana.
- **Chrome prerender**: `document.prerendering === true` fuerza render eager (sin observer) — el page activado al usuario ya está listo.

Decisiones clave

rootMargin 200 px Pre-renderiza el widget justo antes de entrar al viewport. El usuario ve contenido, no spinner. Recomendado por Google Search Central porque no depende de scroll.	Bypass de Googlebot Google Search Central: "Googlebot doesn't scroll, scroll events never fire". <code>IS_BOT</code> salta el observer y renderiza directo, garantizando indexación.
requestIdleCallback timeout 1500 ms	min-height por tipo

El widget no compite con el LCP del host. Si el navegador no está idle a tiempo, el timeout fuerza el render. Fallback a `setTimeout`.

Cada widget reserva su espacio antes de pintar contenido (badge 90px → wall 560px). CLS aportado ≈ 0. Floating omitido (es `position:fixed`).

Método de validación

Página de test sin dependencias: `public/widget-test.html` — se sirve con `npm run dev` en `http://localhost:3000/widget-test.html`.

- Mockea `fetch` a `widget-proxy` y a Google Translate (corre offline).
- Monta 5 widgets a distintas alturas + 1 floating.
- Panel sticky con 10 invariantes evaluadas en vivo: lazy schedule, espacio reservado, eager-init guard, errores JS, CLS aportado, pausa por visibility, bypass de bot.
- Modo `?bot=1` → fuerza `navigator.userAgent` a Googlebot y valida bypass.
- Botón "Ocultar pestaña" → simula `document.hidden` 5 s y compara `transform` de carruseles.

Validación oficial pendiente (manual, post-deploy en staging): Google Search Console → URL Inspection → *Test Live URL* · Rich Results Test · PageSpeed Insights antes/después.

Pendientes para próximos PRs

Sev.	Hallazgo	Por qué
	JSON-LD <code>AggregateRating</code> self-serving en stars-carousel	Google ignora rich result self-serving para LocalBusiness/Organization desde 2019. Se debe servir desde <code>web.opynio.com</code> , no desde el host.
	<code>rel="noopener nofollow"</code> en anchors <code>target="_blank"</code>	Tabnabbing + riesgo de link scheme con miles de hosts apuntando con anchor idéntico.
	Bot-path solo en stars-carousel	Los 8 widgets restantes inyectan reviews completas a Googlebot → riesgo duplicate content entre hosts.
	CSS global con <code>!important</code>	Migrar a Shadow DOM para aislar y eliminar riesgo de pisar al host.
	UX del floating widget	Sin botón cerrar, sin <code>data-position</code> , choca con cookie banners y chats del host.

Archivos modificados / creados

- **Modificado:** `public/widget.js` — nuevo scheduler de lazy loading + helpers + carruseles visibility-aware (v6.0 → v6.1)
- **Nuevo:** `public/widget-test.html` — página de test diagnóstica con mock de fetch y panel de invariantes
- **Nuevo:** `docs/11-WIDGET-LAZY-LOADING-V6.1.md` — documentación técnica completa del cambio